TECHNICAL AND PLAIN-LANGUAGE COMPANION

Explanation Week 3

RabbitMQ work queue, separate WorkerService, bounded retries, and durable dead-letter quarantine on Docker Desktop.

This file explains

- Why the in-process background worker was replaced with a RabbitMQ handoff.
- How the API and `NSA.Worker` form the two-project runtime boundary.
- Why successful work is saved before manual acknowledgement, with only three processing attempts.
- How SQL Server, RabbitMQ Management, API, and WorkerService run together on Docker Desktop.

FLOW

```
HTTP → API → SQL + confirmed RabbitMQ command → WorkerService → SQL → ACK
final reject → DLX → DLQ
```

KEY GUARANTEE

Broker-confirmed work is no longer owned by the API process. RabbitMQ retains it until the WorkerService acknowledges successful processing or deliberately dead-letters it. Delivery remains at least once.

Source baseline: two-project Week 3 implementation at `e17bc39`; runtime evidence at `2b4d98d`. Prepared 26 August 2026.

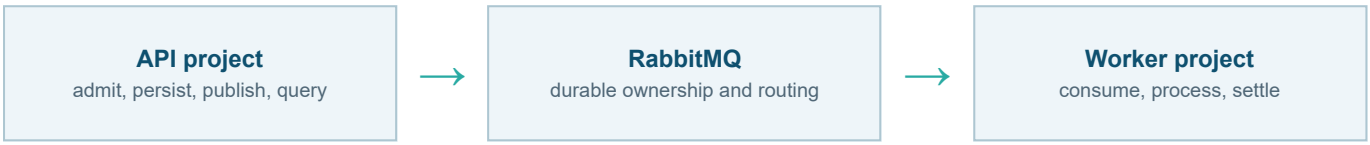
1. Two-project runtime boundary

Source: `e17bc39:notification-api/README.md:5–35` (selected and condensed)

```
 9  There are two deployable processes:
11  API — NSA.csproj
    owns HTTP admission, SQL persistence/status, RabbitMQ publication
12  Worker Service — NSA.Worker.csproj
    owns RabbitMQ consumption, retry, DLQ routing, processing

24  POST bulk → API → SQL job/items commit → RabbitMQ → main queue
26  GET status → API → SQL                              |
28                                                Worker Service
30                              SQL progress ← notification processor
32                                   manual ACK, retry, or DLX → DLQ

35  Broker commands contain identifiers/timestamps, not message content.
```



Line(s)	Technical explanation	Plain language
9–12	The Web API and WorkerService are separate executable projects and process lifetimes.	The receptionist and the workroom can restart or scale independently.
24–32	SQL stores the job and progress; RabbitMQ transfers processing responsibility and routes final rejection to the DLQ.	The database holds the work record; the broker hands the work to the worker and quarantines failures.
35	The versioned command carries job/message/correlation identifiers and time, while recipient, subject, and body remain in SQL.	The queue carries a claim ticket, not the private notification contents.

> **Boundary:** `NSA.Worker` uses a hosted `BackgroundService` internally, but it runs in its own executable and container. The DLQ is a RabbitMQ queue—not a third application project.

2. Entire stack on Docker Desktop

Source: `e17bc39:notification-api/compose.week3.yml:5–120` (selected and condensed)

```
  5  services:
  6    sqlserver:
  7      image: mcr.microsoft.com/mssql/server:2022-latest
 14      volumes: [sqlserver-data:/var/opt/mssql]
 18      healthcheck: ...

 29    rabbitmq:
 30      image: rabbitmq:4.3-management
 35      ports: ["127.0.0.1:15672:15672"]
 36      volumes: [rabbitmq-data:/var/lib/rabbitmq]
 40      healthcheck: ...

 48    api:
 51      dockerfile: Dockerfile.api
 64      depends_on: sqlserver: healthy; rabbitmq: healthy

 81    worker:
 84      dockerfile: notification-api/Dockerfile.worker
 94      depends_on: sqlserver: healthy; rabbitmq: healthy; api: healthy

111  volumes: sqlserver-data; rabbitmq-data
117  networks: week3 → nsa-week3-network
```

Line(s)	Technical explanation	Plain language
6–46	SQL Server and RabbitMQ Management are containers with health probes and persistent named volumes.	The database and queue server live inside Docker and keep their data across normal restarts.
48–79	The API image starts only after SQL and RabbitMQ are healthy and exposes HTTP on loopback port 8080.	The front door waits until both dependencies are usable.
81–109	The WorkerService has its own image, dependency ordering, configuration, and readiness marker.	The workroom is a separate container and reports when it can actually consume.
111–120	Both stores use named volumes; all four services share one named bridge network.	Containers can find one another by service name without installing services on Windows.

> **Local boundary:** Docker Desktop runs all four services. API `8080`, RabbitMQ UI `15672`, and SQL `14330` bind only to localhost; AMQP stays on the internal network at `rabbitmq:5672`. Required credentials come from an ignored `.env` file.

3. Admission and confirmed publication

Sources: `BulkNotificationJobService.cs:65–108` and `RabbitMqBulkNotificationPublisher.cs:19–54,82–97` at `e17bc39` (selected lines)

```
S65  jobs.Add(job);
S66  await jobs.SaveChangesAsync(cancellationToken);
S68  var command = new BulkNotificationRequestedV1(
S69      BulkNotificationRequestedV1.CurrentSchemaVersion,
S70      Guid.NewGuid(),
S71      job.Id,
S72      job.CorrelationId,
S73      now);
S75  try {
S77      await publisher.PublishAsync(command, cancellationToken);
S79  } catch {
S81      job.Status = BulkNotificationJobStatuses.PublishFailed;
S103     throw new BulkNotificationPublishException(...);

P22  var properties = new BasicProperties {
P26      DeliveryMode = DeliveryModes.Persistent,
P27      MessageId = message.MessageId.ToString("D"), ... };
P36  await resiliencePolicies.Policy.ExecuteAsync(async token => {
P41      await channel!.BasicPublishAsync(
P42          options.Value.CommandExchange,
P43          options.Value.CommandRoutingKey,
P44          mandatory: true, basicProperties: properties, body: body);
P92  channel = await connection.CreateChannelAsync(
P94      publisherConfirmationsEnabled: true,
P95      publisherConfirmationTrackingEnabled: true);
```

Line(s)	Technical explanation	Plain language
S65–77	The API commits the job/items, constructs a small versioned reference, then directly publishes it.	Write the work order to the ledger, then hand its ticket to the queue.
P22–29	The JSON message is persistent and correlated; its payload contains identifiers rather than notification content.	The ticket is durable and traceable without copying the private form.
P36–95	Polly bounds transient publish retries/circuit breaking; mandatory routing and publisher confirms detect failed handoff.	Retry brief queue outages, stop hammering a failing broker, and require a receipt.
S79–105	Unconfirmed publication records `PublishFailed` when possible and surfaces a retryable service error instead of claiming acceptance.	No queue receipt means the API does not say the job was accepted.

> **Accepted limit:** the SQL commit and direct RabbitMQ publish are two operations, not one atomic transaction. A crash between them can leave a persisted job without a command; this design does not claim lossless or exactly-once admission.

4. Durable main queue and DLQ topology

Sources: `e17bc39:notification-api/appsettings.json:17-40` and `RabbitMqTopology.cs:12-72` (selected lines)

```
nsa.notifications.commands.v1  (durable direct exchange)
        | bulk.requested.v1
        ▼
nsa.notifications.bulk.v1      (durable quorum main queue)
        | final or non-retryable reject: requeue=false
        ▼
nsa.notifications.dead-letter  (durable direct DLX)
        | bulk.dead-letter.v1
        ▼
nsa.notifications.bulk.dlq     (durable quorum DLQ)
```

```
23  "CommandExchange": "nsa.notifications.commands.v1",
24  "CommandQueue": "nsa.notifications.bulk.v1",
25  "CommandRoutingKey": "bulk.requested.v1",
26  "DeadLetterExchange": "nsa.notifications.dead-letter",
27  "DeadLetterQueue": "nsa.notifications.bulk.dlq",
28  "DeadLetterRoutingKey": "bulk.dead-letter.v1"

T31  ["x-queue-type"] = "quorum",
T32  ["x-delivery-limit"] = options.BrokerDeliveryLimit,
T33  ["x-dead-letter-exchange"] = options.DeadLetterExchange,
T34  ["x-dead-letter-routing-key"] = options.DeadLetterRoutingKey
T36  await channel.QueueDeclareAsync(
T37      options.CommandQueue,
T38      durable: true, ...);
T57  await channel.QueueDeclareAsync(
T58      options.DeadLetterQueue,
T59      durable: true, ...);
```

Line(s)	Technical explanation	Plain language
`23–28`	Stable, versioned names give publishers, consumers, tests, and operators one routing vocabulary.	Every component agrees on the same labeled intake and quarantine bins.
`T31–34`	The main queue is quorum-backed and configured with a DLX/routing key plus a broker delivery-limit safeguard.	The queue is durable and knows exactly where final failures belong.
`T36, T57`	Both the command queue and DLQ are durable, non-exclusive, non-auto-delete queues.	Neither queue disappears just because an app stops.

> **DLQ meaning:** the dead-letter queue retains poison or exhausted commands for inspection in RabbitMQ Management. This two-project design has no automatic replay service; recovery is an explicit operator decision.

5. WorkerService settlement order

Source: `e17bc39:notification.worker/RabbitMqBulkNotificationWorker.cs:41–70,154–175` (selected lines)

```
41  await RabbitMqTopology.DeclareAsync(channel, options.Value, stoppingToken);
42  await channel.BasicQosAsync(
43      prefetchSize: 0,
44      prefetchCount: checked((ushort)options.Value.PrefetchCount),
45      global: false,
46      cancellationToken: stoppingToken);

52  await channel.BasicConsumeAsync(
53      options.Value.CommandQueue,
54      autoAck: false,
55      consumer,
56      stoppingToken);

154 var retryCount = ReadRetryCount(headers);
157 await using var scope = scopeFactory.CreateAsyncScope();
158 var processor = scope.ServiceProvider.GetRequiredService<BulkNotificationProcessor>();
159 var disposition = await processor.ProcessAsync(command.JobId, stoppingToken);
160 if (disposition == BulkNotificationProcessDisposition.DeadLetter) {
162     await channel.BasicRejectAsync(
        eventArgs.DeliveryTag, requeue: false, stoppingToken);
167     return;
168 }
170 await channel.BasicAckAsync(
        eventArgs.DeliveryTag, multiple: false, stoppingToken);
```

Line(s)	Technical explanation	Plain language
41–46	The worker declares the same topology and limits unacknowledged deliveries to configured prefetch 1.	Take one work order at a time so settlement is deliberate and observable.
52–55	`autoAck: false` leaves RabbitMQ responsible until application code settles the delivery.	Receiving a job is not the same as finishing it.
157–170	`ProcessAsync` persists item progress and terminal job state before it returns; only then does the worker ACK.	Update the ledger before telling the queue to discard its copy.
160–167	A delivery for an already dead-lettered job is rejected without requeue and follows the configured DLX path.	Do not run quarantined work again by accident.

> **Failure order:** if the worker stops before ACK, RabbitMQ can redeliver the command. That favors retaining work over silently losing it, but a repeated side effect is possible—hence the explicit at-least-once guarantee.

6. Bounded retry and poison handling

Source: `e17bc39:notification.worker/RabbitMqBulkNotificationWorker.cs:124–150,181–225` (selected lines)

```
124 var body = eventArgs.Body.ToArray();
129 command = JsonSerializer.Deserialize<BulkNotificationRequestedV1>(
        body, SerializerOptions);
131 catch (JsonException exception) {
134     await channel.BasicRejectAsync(
        eventArgs.DeliveryTag, requeue: false, stoppingToken);
135     return;
136 }
138 if (command is null
139     || command.SchemaVersion != BulkNotificationRequestedV1.CurrentSchemaVersion
140     || command.MessageId == Guid.Empty
141     || command.JobId == Guid.Empty
142     || string.IsNullOrWhiteSpace(command.CorrelationId)
143     || !string.Equals(eventArgs.BasicProperties.Type,
            BulkNotificationRequestedV1.MessageType, StringComparison.Ordinal)) {
150     await channel.BasicRejectAsync(
        eventArgs.DeliveryTag, requeue: false, stoppingToken);
151     return;
152 }

183 var completedAttempts = retryCount + 1;
184 if (completedAttempts >= options.Value.MaxDeliveryAttempts) {
186     await MarkDeadLetteredAsync(command.JobId, completedAttempts, stoppingToken);
194     await channel.BasicRejectAsync(
        eventArgs.DeliveryTag, requeue: false, stoppingToken);
195     return;
196 }
198 await RecordRetryAsync(command.JobId, completedAttempts, stoppingToken);
199 var retryProperties = CreateRetryProperties(
        eventArgs.BasicProperties, completedAttempts);
202 await channel.BasicPublishAsync(
203     options.Value.CommandExchange,
204     options.Value.CommandRoutingKey,
205     mandatory: true,
206     basicProperties: retryProperties,
207     body: body,
208     cancellationToken: stoppingToken);
209 await channel.BasicAckAsync(
        eventArgs.DeliveryTag, multiple: false, stoppingToken);
218 catch (Exception publishException) when (!stoppingToken.IsCancellationRequested) {
224     await channel.BasicRejectAsync(
        eventArgs.DeliveryTag, requeue: true, stoppingToken);
225 }
```



Line(s)	Technical explanation	Plain language
124–151	Malformed JSON, missing identifiers, unsupported schema, or wrong AMQP type is non-retryable and rejected directly to the DLQ.	Repeating invalid bytes cannot repair them, so quarantine them immediately.
183–195	Retryable failures stop after three total attempts; SQL records `DeadLettered` before final rejection.	Try a limited number of times, record the result, then stop the loop.
198–224	The replacement delivery is confirm-published before ACKing the original. If publish fails, the original is requeued.	Do not throw away the old ticket until the new retry ticket is safely accepted.

Review summary

Week 3 boundaries, evidence, and short answers

Failure behavior

- API publish cannot be confirmed: record `PublishFailed` when possible and return a retryable service error.
- Worker stops before ACK: RabbitMQ redelivers; persisted SQL status remains queryable, but duplicate effects remain possible.
- Malformed or exhausted command: reject without requeue; the main queue's DLX routes it to the durable DLQ.
- SQL commit/direct publish and retry-publish/ACK are non-atomic; no automatic DLQ replay or exactly-once claim exists.

Review answers

- RabbitMQ replaces process-local queue ownership with durable, inspectable broker ownership.
- The separate WorkerService isolates API capacity and lifecycle while keeping job progress in shared SQL.
- Manual ACK is last because the broker must retain responsibility until progress is durable.
- The DLQ is a RabbitMQ quarantine queue within the two-project solution, not another worker application.

Executable evidence

- The recorded delivery passed 76/76 tests, built both application images, and brought SQL Server, RabbitMQ, API, and WorkerService to healthy state in Docker Compose.
- Runtime records cover HTTP acceptance through completion, stop-before-ACK redelivery, in-flight broker restart, unavailable publication, invalid commands, three poison attempts, identifier correlation, and RabbitMQ Management DLQ inspection.

The detail pages use the final two-project source cutoff; the verification summary uses the original captured Week 3 runtime evidence. Later runtime components and guarantees are intentionally excluded.